

Algorísmia QP 2017–2018

Una solució possible del Examen final

15 de juny de 2018

Durada: 2h 50m

Instruccions generals:

- Els exercicis 1 i 2 s’han de resoldre fent servir l’espai reservat per a cada resposta.
- Heu d’argumentar la correctesa i l’eficiència dels algorismes que proposeu. Per això podeu donar una descripció d’alt nivell de l’algorisme amb les explicacions i aclariments oportuns que permetin concloure que l’algorisme és correcte i té el cost indicat.
- Heu de justificar totes les vostres afirmacions, en cas contrari la nota de la pregunta serà 0.
- Podeu fer crides a algorismes que s’han vist a classe, però si la solució és una variació n’haureu de donar els detalls.
- Es valorarà especialment la claredat i concisió de la presentació.
- Entregueu per separat les vostres solucions de cada bloc d’exercicis (Ex 1, Ex 2, Ex 3 i Ex 4).
- La puntuació total d’aquest examen és de **10 punts**.

Exercici 1 (3 punts) Digueu si les següents afirmacions són certes o no.

- (a) (0.5 punts) Donats n enters, cadascun amb un valor màxim de n^{100} , es poden ordenar en temps lineal.

Sol. CERT. Recordeu que la complexitat de RADIX és $O((n+b)d)$ on d és en nombre de dígit per cada enter, b és la base per a representar els enters, i n és el nombre d'enters que ve donat pel problema. Notem que com els enters van de 0 fins a $n^{100} - 1$, tenim $d = \log_b n$. Volem fer que $d = \Theta(1)$ per tant necessitem $b = n$, el que donara una complexitat de $200n = \Theta(n)$. Si tenim el cas n^{100} restem 1 del valor de cada enter, estarem al cas previ i quan els tenim ordenats, sumem 1 a cada enter.

- (b) (0.5 punts) Per a $r > 0$ definim un r -camí com un camí en què totes les arestes del camí tenen pes $< r$. Si G conté un r -camí del node s al node t , llavors tot MST de G ha de contenir un r -camí de s a t .

Sol. CERT. Supongamos que es cierta. Si no hay un r -camino en el MST de G al menos una de las aristas e en el camino de s a t en el MST tiene peso mayor que r . Si la quitamos creamos dos árboles, uno contiene s y el otro t . Además sabemos que hay un r -camino de s a t , por lo tanto entre las aristas que conectan los dos árboles hay una con peso $\leq r$, añadiéndola tenemos un árbol de expansión con peso menor. Lo que es una contradicción..

- (c) (0.5 punts) Donat un graf dirigit $G = (V, E)$ amb pesos sobre les arestes $w : E \rightarrow \mathbb{Z}^+$, l'algorisme de Dijkstra i l'algorisme de Bellman-Ford poden produir diferents arbres de camins mínims, malgrat que la mínima distancia entre dos vèrtexs qualsevol serà la mateixa.

Sol. CERT. Mirar les transparencies.

- (d) (0.5 punts) Sigui $G = (V, E)$ un graf dirigit amb pesos sobre les arestes $w : E \rightarrow \mathbb{Z}^+$ i siguin $c, a \in V$. Aleshores el camí més curt de c a a no canvia quan incrementem en 1 el pes de totes les arcs.

Sol. FALS. Considereu un graf amb la aresta (c, a) amb pes 4 i un camí de longitud 3 de c a a amb totes les arestes amb pes 1 (total 3). A l'incrementar en 1, tenim el camí d'una aresta amb pes 5 i el de tres amb pes total 6.

- (e) (0.5 punts) El temps per tal de resoldre un problema de programació dinàmica sempre és $\Theta(k)$, on k és el nombre de subproblemes diferents del problema.

Sol. FALS. Hem vist molts problemes que requereixen cost no constant per subproblema.

- (f) (0.5 punts) Si totes les capacitats a una xarxa de flux són múltiples de 5, aleshores el flux màxim també tindrà un valor que serà un múltiple de 5.

Sol. CERT. L'algorisme de Ford-Fulkerson incrementa el flux per la capacitat mínima del camí al graf residual. Com que totes les capacitats son múltiples de 5, aquesta també ho es i el valor del flux resultant també ho serà.

Exercici 2 (3 punts)

- (a) (1.5 punts) Sigui $G = (V, E)$ un graf dirigit amb pesos $w : V \rightarrow \mathbb{Z}^+$, donats dos vèrtexs $u_1, u_2 \in V$ definim el pes d'un camí $w(P(u_1, u_2))$ com $\sum_{v \in P(u_1, u_2)} w(v)$. Si tenim com a entrada G, w, u_1, u_2 , doneu un algorisme per a calcular el camí amb menys pes entre u_1 i u_2 . Quina és la seva complexitat? (Ajut: transformeu G en G' on els pesos siguin a les arestes)

Sol. Moltes solucions possibles. La que se indica a l'ajut: construir un nou graf G' idèntic al G , excepte que per a tota aresta (u, v) definim $w(u, v) = w(v)$. El cost de crear G' és $O(n+m)$. Com els pesos són positius, podem utilitzar Dijkstra per calcular el camí més curt entre u_1 i u_2 amb un cost $O(m \lg n)$ (utilitzant un heap) o podem utilitzar Bellman-Ford amb un cost $O(nm)$. Per a demostrar que el camí més curt a G' també és el camí més curt a G , sigui $u_1, v_1, \dots, v_k, u_2$ un camí amb pes $w(u_1) + w(v_1) + \dots + w(v_k) + w(u_2)$ a G , el mateix camí a G' tindrà pes $w(u_1, v_1) + \dots + w(v_k, u_2)$ que és $= w(v_1) + \dots + w(v_k) + w(u_2) \Rightarrow$ els pesos de qualsevol camí en G i G' es diferencien en $w(u_1)$, per tant un camí amb distància mínima a G' també serà un camí amb distància mínima a G .

- (b) (1.5 punts) Resoleu el següent problema: Volem anar de Barcelona a Paris seguint l'autopista a través de Lyon, i tenim n benzineres $B_1 \dots, B_n$ al llarg d'aquesta autopista. Quan teniu el dipòsit ple, el vostre cotxe pot recórrer K km. La benzinera B_1 és a Barcelona i la benzinera B_n és a Paris. La distància entre benzineres consecutives és sempre menor que K , és a dir $d(B_i, B_{i+1}) < K$,

Doneu un algorisme que ens proporcioni una estratègia per fer el mínim nombre d'aturades a benzineres al llarg del viatge Barcelona-Paris.

Solució: L'estratègia correcta és esperar el màxim per a carregar benzina. Fem el següent algorisme voraç:

```
d := 0; j := 1
for i = 1 to n - 1 do
  if d + di+1 > k then d := 0, j := j + 1 fi
  d := d + di+1
od
return j
```

Si aquest algorisme torna j aturades, però l'òptim és aturar-se a $S_{i_1}, \dots, S_{i_m}$ per $m < j$ i sigui S_{i_l} la primera benzinera després que la j -èsima aturada de l'algorisme, aleshores el cotxe es quedaria sense benzina entre $S_{i_{j-1}}$ i S_{i_j} , contradicció. La complexitat de l'algorisme és $O(n)$.

Exercici 3 (2 punts) Considereu el problema d'emmagatzemar n llibres als prestatges de la biblioteca. L'ordre dels llibres és fixat pel sistema de catalogació i, per tant, no es pot canviar. Els llibres han d'aparèixer a les prestatgeries en l'ordre designat. Les

prestatgeries d'aquesta biblioteca tenen amplada L i són regulables en alçada. Un llibre b_i , on $1 \leq i \leq n$, té gruix t_i i alçada h_i . Una vegada es decideix quins llibres es fiquen a un prestatge s'ajusta la seva alçada a la del llibre més alt que col·loquem al prestatge. Doneu un algorisme que ens permeti col·locar els n llibres a les prestatgeries de la biblioteca de manera que es minimitzi la suma de les alçades dels prestatges utilitzats.

Solució: Vamos a desarrollar una solución vía programación dinámica. Si todos los libros caben en un estante ponerlos en uno es óptimo. Si no caben, miramos una solución óptima y consideramos el primer libro i que no está en el primer estante. El coste de la solución es la altura del primer estante $\max_{1 \leq j < i} h_j$ mas la altura de la solución óptima de los libros i a n . Con esta estructura de suboptimalidad obtenemos la siguiente recurrencia. Utilizo $T(i)$ como altura óptima de colocar en estantes libros $i \cdots n$ y $C(i, j)$ indica si la anchura de los libros i a j , $i \leq j$ cabe o no en un estante.

$$T(i) = \min_{j: C(i, j)} \{ \max_{1 \leq j \leq i} h_j + T(j+1) \}$$

Podemos precalcular C usando PD en tiempo $O(n^2)$. Después calcular T en tiempo lineal por elemento.

Para obtener la solución óptima añadiremos la estructura de punteros necesaria para almacenar el último libro en el primer estante de $T(i)$.

El coste total del algoritmo es $O(n^2)$ y utilizamos espacio $O(n^2)$.

Exercici 4 (2 punts) Les xarxes ad-hoc, composades per dispositius sense fils de baixa potència, s'han proposat per situacions com els desastres naturals en què els coordinadors dels treballs de rescat podrien controlar les condicions en zones de difícil accés. La idea és que una gran col·lecció d'aquests dispositius sense fils es podria llançar des d'un avió en una regió per a continuació reconfigurar-se com una xarxa operativa. Estem parlant de: (a) dispositius relativament barats, el quals (b) es llancen des d'un avió a (c) un territori perillós; i per la combinació de (a), (b) i (c), es fa necessari fer front a la fallida d'un nombre raonable dels dispositius. Cada dispositiu té un abast de transmissió limitat, pot comunicar-se amb altres dispositius que es troben com a màxim a r metres d'ell.

Ens agradaria poder determinar si hi han suficients camins que no comparteixen arcs entre els dispositius de dues zones diferents, A i B , a fi de garantir que un senyal d'emergència pot arribar sense problemes des d'algun dispositiu a A a algun dels dispositius a B . Per tal d'evitar interferències volem, a més, controlar el *nivell de participació* dels dispositius. Aquest nivell és el màxim nombre de camins en què un dispositiu hi participa.

Suposeu que després del llançament podem obtenir les coordenades $p_i = (x_i, y_i)$ dels n dispositius operatius que formen la xarxa inicial. Suposeu també, que les regions A i B són rectangles i que us donen les coordenades de les seves cantonades. Finalment,

us donen també un valor c que determina el nombre de camins mínim que voldríem tenir entre A i B .

Dissenyeu un algorisme per determinar si es possible trobar c camins que no comparteixin arestes entre A i B .

Solució: Hemos visto en clase como calcular el máximo número de caminos que no comparten aristas entre dos vertices poniendo capacidad 1 en los arcos. Aquí nos piden calcular este número pero entre dos conjuntos. Para ello añadiremos dos vértices s y t . Conectaremos s con todos los dispositivos en la zona A y conectaremos todos los dispositivos de la zona B con t . Añadimos un arco (a, b) si el dispositivo b está dentro del rango de a . Obtenemos una red de flujo adecuada añadiendo capacidad 1 a todas las conexiones entre dispositivos, ya que solo se pueden utilizar una vez. Las conexiones desde s o hacia t se pueden usar más veces, ya que corresponden con pasos por uno de los vértices, pero estos no tienen límite. Podemos asignarles capacidad ∞ .

Una vez obtenido el flujo con valor máximo comprobamos que su valor sea o no mayor o igual que c .

Si utilizamos Ford-Fulkerson, podemos para la ejecución la primera vez que el valor del flujo sea $\geq c$. Esto nos da coste $O(cn)$. Si utilizamos Edmons-Karp el coste es $O(n^2m)$. Como el número total de caminos puede ser exponencial c puede ser un número muy grande. Para garantizar coste polinómico usaremos el segundo análisis de coste.